

Module 16: STL Containers & Iterators

Introduction to C++ Programming — Part 2: Intermediate C++

Module Overview

Students already trust `std::vector` deeply by this point in the course - use that trust as leverage. Frame `map` and `set` as 'the same family, different lookup strategy' rather than unrelated new tools, and use iterators to reveal what the range-based for loop (Module 5) was quietly doing all along.

Suggested Duration: 75 minutes

Learning Objectives

By the end of this module, students will be able to:

- Use `std::map` to store and look up values by a key instead of a position.
- Explain the `map[]` existence-check gotcha and use `.count()/.find()` to avoid it.
- Use `std::set` to maintain a collection with automatically-enforced uniqueness.
- Explain what a range-based for loop is doing underneath, using explicit iterators.
- Erase an element from a vector while iterating, without invalidating the loop.

Prerequisites

- Module 5 (vector, range-based for) from Part 1 - this module is a direct extension of both.

Materials & Instructor Prep

- `16-stl-containers/examples/01_map_basics.cpp` preloaded, plus a second terminal/tab ready to show `std::vector`'s iterator loop from `examples/03_iterators.cpp` for direct side-by-side comparison.

Key Vocabulary

Term	Definition
<code>std::map<K, V></code>	A key/value container, automatically kept sorted by key.
<code>std::set<T></code>	A container of unique values with no duplicates, automatically kept sorted.

Term	Definition
Iterator	An object that points to one container element and knows how to move to the next - the general mechanism behind every STL loop.
.begin() / .end()	An iterator to the first element, and a boundary marker just past the last element (never dereferenced directly).
auto	Lets the compiler infer a variable's type - useful for verbose types like iterators, still used sparingly by preference in this course.

Lesson Outline & Instructor Talking Points

1. Motivate map from vector's limitation (15 min)

- Ask: 'in the Part 1 capstone, how did findStudent(name) actually search for a student?' (a loop scanning every element). Ask: 'what if there were 10,000 students?' - motivate map as 'lookup by name directly, without scanning.'
- Live-build componentValues from examples/01_map_basics.cpp. Point out map<K, V> is a template with TWO type parameters, extending the single-parameter vector<T> pattern they already know.

2. The [] existence-check gotcha (15 min)

- Ask the class to predict: 'if I write componentValues["R2"] to CHECK whether R2 exists, and it doesn't, what happens?' Most will guess it just returns something like 'not found.'
- Reveal: it silently CREATES the key with a default value. Demonstrate live, then introduce .count() as the safe existence check.

Instructor note: *This gotcha is genuinely surprising and worth a real pause - it's exactly the kind of silent, no-error bug this course has flagged repeatedly (Module 2's integer division, Module 6's pass-by-value) as more dangerous than a loud crash.*

3. Looping over a map (10 min)

- Show the range-based for (const auto& pair : componentValues) loop, explaining pair.first/pair.second. Note the free bonus: a map always iterates in sorted-by-key order, with no extra work.

4. std::set (10 min)

- Run examples/02_set_basics.cpp. Keep this brief - the core idea (automatic duplicate rejection) is simple once map is already understood.

5. Iterators - unmasking the range-based for loop (20 min)

- Ask directly: 'when you write `for (int score : scores)`, what is C++ actually doing underneath?' Let the class guess before revealing.
- Run `examples/03_iterators.cpp`, showing the explicit `begin()/end()/++it` loop produces IDENTICAL output to the range-based version. State plainly: the compiler translates one into the other - now they know what was hidden.
- Explicitly connect `*it` (dereferencing an iterator) back to Module 10's `*ptr` (dereferencing a pointer) - name the fact that a raw pointer IS a valid iterator for a plain array, tying the two modules together.

6. Erasing while iterating - a real gotcha (10 min)

- Run `examples/04_erase_while_iterating.cpp`. Point out the loop header has NO `++it` - ask why, before explaining that `erase()` invalidates the iterator you just used and returns the next valid one instead.

Instructor note: *Keep this brief and clearly labeled as 'recognize this pattern, don't necessarily derive it from scratch under time pressure' - it's a real but narrow gotcha.*

Common Student Misconceptions

Misconception	How to Address It
"Using <code>[]</code> to check whether a key exists in a map is safe."	It silently CREATES the key with a default value if missing - use <code>.count()</code> or <code>.find()</code> to check existence without side effects.
"A range-based for loop and an explicit iterator loop are fundamentally different mechanisms."	The range-based form is exactly the iterator loop, compiler-generated for you - show them side by side producing identical output.
"You can safely keep using an iterator after <code>erase()</code> -ing the element it points to."	<code>erase()</code> invalidates that iterator; always use the iterator <code>erase()</code> itself returns to continue safely.

Exercises & Module Project

Exercise: `16-stl-containers/exercises/exercise1.cpp` - a word-count map plus a unique-words set.
Project (three tracks): Generic - Inventory Tracker; EE - Component Inventory (voltage samples by node); Biomedical - Patient Database (readings by patient ID). All three build a map-based database keyed by name/ID, with a flagging rule applied while iterating.

Facilitation guidance: In the EE and Biomedical tracks, watch specifically for correct use of `.back()` to check a container's MOST RECENT sample/reading (as specified) rather than accidentally checking the first or averaging when a single latest value was asked for - a common small slip under this project's requirements.

Discussion & Comprehension Check Questions

1. Why does looping over a `std::map` always visit keys in sorted order, with no `sort()` call needed?

Guidance: A map maintains sorted order internally as an ongoing property of the container itself, not as a one-time operation you have to remember to run.

2. What does a range-based for loop actually compile down to?

Guidance: An explicit iterator loop using `.begin()`, `.end()`, and `++it`, exactly as demonstrated side by side in class.

Assessment Ideas

- Given a short scenario ('look up a patient's most recent reading by ID' vs. 'store an ordered list of doses given today'), have students choose map vs. vector and justify the choice in one sentence.

Answer Key Notes

See `16-stl-containers/exercises/solutions/exercise1_solution.cpp`.

Wrap-Up & Bridge to Next Module

Bridge forward: "You've been hand-writing loops to find a maximum, count matches, and convert values since Module 5. Next module replaces those hand-written loops with ready-made STL algorithms, plus lambdas to customize them on the spot."

Instructor Tips

- If a student already knows Python dictionaries or JavaScript objects, map will feel immediately familiar - use that prior knowledge as a bridge rather than re-explaining key/value pairs from zero for that student.